

Dokumentacja Projektowa

System Zarządzania Harmonogramem i Komunikacją
dla Międzynarodowego Festiwalu Kolęd i Pastoralek (MFKiP) w Będzinie

Piotr Gwioździk
Wydział Fizyki i Informatyki Stosowanej, AGH
Zaawansowane Technologie Internetowe

7 czerwca 2026

Spis treści

1	Wstęp i Cel Projektu	2
2	Architektura Systemu i Technologie	2
2.1	Stos Technologiczny	2
3	Struktura Logiczna API (Backend)	2
3.1	Zarządzanie Uwierzytelnianiem (AuthController)	2
3.2	Zarządzanie Dniami Festiwalowymi (DayController)	2
3.3	Harmonogram Występów (PerformanceController)	3
4	Architektura i Implementacja Frontendu (React)	3
4.1	Architektura Komponentów i Stanu	3
4.2	Integracja z WebSockets i Reaktywność	3
4.3	Dedykowane Panele i Widoki Klasyfikowane Rolami	3
5	Zabezpieczenia i CORS	4
6	Jakość Oprogramowania i Testy	4
7	Instrukcja Użytkownika i Dewelopera	5
7.1	Wymagania Środowiskowe	5
7.2	Kompilacja i Uruchamianie	5
7.3	Uruchamianie Testów	5
7.4	Generowanie Dokumentacji Javadoc	5

1 Wstęp i Cel Projektu

Niniejsza dokumentacja opisuje architekturę oraz implementację systemu informatycznego zrealizowanego na potrzeby **Międzynarodowego Festiwalu Kolęd i Pastorałek (MFKiP) w Będzinie**.

Głównym celem systemu jest automatyzacja zarządzania harmonogramem przesłuchań i występów, ewidencja wolontariuszy oraz, co kluczowe, dostarczenie niezawodnej komunikacji w czasie rzeczywistym między różnymi strefami organizacyjnymi (takimi jak recepcja uczestników, reżyserka sceny oraz panel administracyjny). System eliminuje opóźnienia informacyjne podczas trwania wydarzenia na żywo.

2 Architektura Systemu i Technologie

System został zaprojektowany w architekturze klient-serwer, wprowadzając całkowity rozdział warstwy prezentacji od logiki biznesowej poprzez bezstanowe API.

2.1 Stos Technologiczny

- **Backend:** Java 17, Spring Boot 4.0.4 (oparty na Spring Framework 7.x).
- **Baza Danych:** PostgreSQL 16. W środowisku programistycznym wykorzystywany jest tunel SSH (2gwiozdzik@taurus port 5433) do połączenia z serwerem bazodanowym.
- **Frontend:** Aplikacja webowa stworzona w oparciu o ekosystem **React** oraz framework **Next.js**.
- **Komunikacja Real-Time:** WebSockets obsługiwane za pomocą protokołu STOMP na poziomie aplikacji.

3 Struktura Logiczna API (Backend)

Backend (`zti.backend`) dostarcza architekturę RESTful API podzieloną na wyspecjalizowane kontrolery.

3.1 Zarządzanie Uwierzytelnianiem (`AuthController`)

Aplikacja wykorzystuje dedykowany system autoryzacji oparty na rolach, unikając ciężkich mechanizmów sesyjnych. Zdefiniowano trzy profile dostępowe, których hasła wstrzykiwane są z pliku `application.properties`:

- **reception** – Przeznaczony dla obsługi punktu rejestracji, umożliwiający podgląd i edycję statusów przybycia.
- **stage** – Dedykowany dla obsługi technicznej i akustyków do śledzenia bieżącego występu.
- **admin** – Pełny dostęp modyfikacyjny do wszystkich zasobów bazy.

3.2 Zarządzanie Dniami Festiwalowymi (`DayController`)

Moduł odpowiedzialny za cykl życia dni festiwalowych (encja `Day`). Zaimplementowano możliwość dynamicznego przełączania aktywnego dnia (`PATCH /api/days/{id}/activate`), co automatycznie filtruje globalny harmonogram występów.

3.3 Harmonogram Występów (PerformanceController)

Centralny punkt logiki biznesowej. Obsługuje encję **Performance**.

- **Transakcyjne Przesunięcia (Reorder):** Endpoint `PATCH /{id}/reorder` pozwala na przesuwanie występów w dół lub w górę harmonogramu za pomocą modyfikacji czasu rzeczywistego w pojedynczej transakcji bazodanowej.
- **Rozgłaszanie Zmian:** Każda akcja modyfikująca dane automatycznie wysyła sygnał `UPDATE` przez `WebSocket`, co wymusza natychmiastowe odświeżenie danych u wszystkich zalogowanych klientów.

4 Architektura i Implementacja Frontendu (React)

Warstwa prezentacji systemu została zaprojektowana jako nowoczesna aplikacja typu Single Page Application (SPA) osadzona w środowisku `Next.js`, co pozwala na zachowanie modułowości komponentów i optymalne zarządzanie stanem.

4.1 Architektura Komponentów i Stanu

Frontend komunikuje się asynchronicznie z serwerem `Spring Boot` pod adresem `http://localhost:8081` za pomocą klienta `HTTP` (np. `Axios` lub natywnego `Fetch API`). Interfejs został podzielony na reaktywne komponenty modułowe, w tym:

- **Tabela Harmonogramu (Schedule Table):** Dynamicznie renderowany widok przeszukań dla wybranego dnia festiwalowego, reagujący na zmiany w czasie rzeczywistym.
- **Moduł Autoryzacji (Auth Context):** Globalny dostawca stanu (`Context API`) przechowujący informacje o bieżącej roli użytkownika (`reception`, `stage`, `admin`) po weryfikacji formularza logowania.

4.2 Integracja z WebSockets i Reaktywność

Kluczowym elementem frontendu jest mechanizm subskrypcji zdarzeń czasu rzeczywistego. Za pomocą bibliotek klienckich protokołu `STOMP` (np. `@stomp/stompjs`), aplikacja inicjalizuje stałe połączenie z endpointem `/ws/**` backendu.

- Po pomyślnym nawiązaniu połączenia, frontend subskrybuje kanał wiadomości `/topic/performances`.
- W momencie odebrania surowego komunikatu tekstowego `"UPDATE"`, komponenty tabeli wyzwalają automatyczne i bezwykopowe ponowne pobranie danych (*re-fetch*) z endpointów `REST` (np. `/api/performances/day/{dayId}`).
- Zapewnia to natychmiastową aktualizację widoków u wszystkich użytkowników systemu bez konieczności odświeżania strony w przeglądarce.

4.3 Dedykowane Panele i Widoki Klasyfikowane Rolami

W zależności od zalogowanej roli, interfejs użytkownika adaptuje swoje możliwości i strukturę:

- **Panel Recepcji (reception):** Koncentruje się na rejestracji wykonawców. Udostępnia szybkie przełączniki statusów przybycia i gotowości zespołu. Modyfikacje te są natychmiastowo przekazywane na zaplecze festiwalu.

- **Panel Reżyserki i Sceny (stage):** Uproszczony, czytelny widok o wysokim kontraście, dostosowany do warunków panujących przy scenie. Prezentuje aktualnie trwający występ, odliczanie czasu oraz zespół oczekujący w kolejce, bez możliwości modyfikacji struktury bazy danych.
- **Panel Administratora (admin):** Pełny interfejs zarządczy. Udostępnia zaawansowane formularze dodawania występów i przerw technicznych, usuwanie pozycji, a także przyciski interaktywne (góra/dół) wywołujące operację **reorder** do natychmiastowej, bezbłędnej reorganizacji kolejności przesłuchań.

5 Zabezpieczenia i CORS

Wprowadzono scentralizowaną konfigurację zabezpieczeń w klasie **SecurityConfig**:

- Konfiguracja **CorsConfigurationSource** jawnie akceptuje żądania wyłącznie z adresu **http://localhost:5174**, zezwalając na pełen zbiór metod HTTP i przekazywanie poświadczeń (**allowCredentials**).
- Zrezygnowano z mechanizmu CSRF, logowania formularzowego oraz **httpBasic**, udostępniając otwarte i bezstanowe endpointy **/api/**** i **/ws/****.

6 Jakość Oprogramowania i Testy

Projekt podlega testom automatycznym opartym o biblioteki **JUnit 5** oraz **Mockito**.

- **Izolacja testów (Standalone Setup):** Kontrolery są testowane przy użyciu **MockMvcBuilders.standaloneSetup()**, co gwarantuje izolację testów. Do konfiguratora wstrzykiwane są atrapy wartości placeholderów za pomocą **.addPlaceholderValue()**, co eliminuje błędy typu **PlaceholderResolutionException**.
- **Deserializacja Dat:** Zintegrowano **JavaTimeModule** w konfiguracji JSON na potrzeby sprawdzania poprawności operowania na obiektach typu **LocalTime**.

7 Instrukcja Użytkownika i Dewelopera

Poniższa sekcja zawiera instrukcje dotyczące uruchamiania, testowania oraz ogólnego korzystania z aplikacji w środowisku deweloperskim.

7.1 Wymagania Środowiskowe

Aby poprawnie uruchomić projekt, system musi spełniać następujące wymagania:

- **Java Development Kit (JDK):** Wersja 17 lub nowsza.
- **Baza Danych:** PostgreSQL 16. Należy upewnić się, że uruchomiony jest kontener bazodanowy lub aktywowano tunel SSH (`ssh -L 5433:localhost:5432 2gwiozdzik@taurus`).

7.2 Kompilacja i Uruchamianie

Aplikacja wykorzystuje narzędzie Maven (wbudowany wrapper `mvnw`), co zwalnia z konieczności globalnej instalacji Mavena w systemie operacyjnym.

1. Otwórz terminal w głównym katalogu projektu (`backend`).
2. Wyczyść poprzednie pliki tymczasowe i skompiluj projekt poleceniem:

```
.\mvnw clean compile
```

3. Uruchom aplikację za pomocą wbudowanego pluginu Spring Boot:

```
.\mvnw spring-boot:run
```

Aplikacja po pomyślnym starcie nasłuchuje na porcie 8081.

7.3 Uruchamianie Testów

Wszystkie zaimplementowane testy jednostkowe można uruchomić masowo z poziomu terminala:

```
.\mvnw test
```

Testy można również uruchamiać bezpośrednio ze środowiska IntelliJ IDEA (klikając prawym przyciskiem myszy na folder `src/test/java` i wybierając **Run 'All Tests'**).

7.4 Generowanie Dokumentacji Javadoc

Kod źródłowy zawiera rozbudowane komentarze zgodne ze standardem Javadoc. Aby wygenerować z nich interaktywną stronę internetową (HTML), wykonaj polecenie:

```
.\mvnw javadoc:javadoc
```

Wygenerowana dokumentacja będzie dostępna w katalogu:
`target/site/apidocs/index.html`.